LAB 3: MQTT & ESP32

José Paca, *UTEC,* Daniel Casquino, *UTEC,* Nicolas Stigler, *UTEC,* Sofia Ku, *UTEC*

I. Introduction

THIS laboratory approaches the connection between data acquisition on an ESP32 and message distribution using MQTT. The experiments cover connectivity, telemetry, control and serial integration.

A. Theoretical Framework

MQTT uses the publisher–subscriber model: a client publishes to a topic, and the server distributes messages according to subscriptions. MQTT 3.1.1 requires an ordered, bidirectional transport, like TCP. Its QoS level 0 offers delivery at most once; it does not guarantee that every message will reach the receiver [1]. Therefore, simply observing a connection is not enough to quantify message loss or receipt of sensor readings.

In the Arduino-ESP32 API, `analogRead` returns a raw, uncalibrated ADC code, while `analogReadMilliVolts` provides a calibrated result in millivolts [2]. An ADC code obtained from a divider with an LDR does not directly equate to illuminance in lux. The relation with illumination requires characterizing the sensor, circuit, and reference conditions; this conversion is not performed here.

PubSubClient 2.8 selects MQTT 3.1.1 by default. Its overload `publish(topic,payload)` constructs a message without retention and without QoS bits 1/2; `loop()` handles client communication [3]. These properties describe the library and allow interpretation of the CP2 sketch; they do not replace a receiver capture.

B. State of the Art

MQTT is commonly studied as a lightweight application-layer protocol for resource-constrained IoT devices. Al-Fuqaha *et al.* identify publish–subscribe protocols, including MQTT, as an important part of the IoT protocol stack and relate protocol selection to device constraints, network conditions, and application requirements [4]. This perspective is relevant to the present experiment because the ESP32 acts as a constrained sensing client while the broker separates the publisher from the monitoring application.

Several studies compare MQTT with alternatives rather than evaluating it in isolation. Thangavel *et al.* evaluated MQTT and CoAP through a common middleware and showed that their behavior depends on the communication pattern and transport assumptions [5]. Similarly, the experimental study by Borsatti *et al.* examined MQTT performance in IoT-to-cloud communication and reported that the selected QoS level affects latency and application behavior [6]. These results support documenting the protocol parameters instead of treating a successful connection as a complete performance evaluation.

For low-power wireless sensor networks, Hunkeler, Truong, and Stanford-Clark proposed MQTT-S as an adaptation of the MQTT model to networks where conventional TCP connectivity is not always available [7]. More recent work by Palmese, Redondi, and Cesana proposed adaptive QoS control for MQTT-S and evaluated its effect on delay and packet errors [8]. In contrast, this laboratory uses standard MQTT over Wi-Fi/TCP and QoS 0 through PubSubClient; therefore, the experiment demonstrates basic telemetry but does not claim the reliability or energy behavior of MQTT-S.

Recent measurements also investigate alternatives to TCP. Jeddou *et al.* characterized delay and energy consumption for MQTT over TCP and QUIC using commercial off-the-shelf devices [9]. Their results reinforce that latency and energy depend on the complete transport stack and hardware, not only on the MQTT message format. The present work is consequently positioned as an implementation and functional validation of an ESP32–MQTT telemetry path. It provides evidence of acquisition and delivery, but it does not estimate lux, compare protocols statistically, or measure energy consumption.

II. Methodology

A. Components and CP2 configuration

The CP2 implementation uses an ESP32 development board, an LDR, a fixed resistor, a breadboard, jumper wires, and a USB cable. The LDR and the fixed resistor form a voltage divider. The divider output is connected to GPIO35, which is used as an ADC input by the ESP32. The exact resistance value and the orientation of the divider must match the physical circuit demonstrated during the checkpoint; changing either one changes the direction and scale of the ADC response.

The ESP32 is programmed with the Arduino IDE and the ESP32 board support package. The sketch uses the `WiFi.h` library for network connectivity and PubSubClient 2.8 for MQTT communication. The serial monitor is configured for 115200 baud. Network credentials are stored in `secrets_local.h`, which is intentionally excluded from the repository. The broker address is supplied through `mqtt_broker` and the broker listens on the standard non-TLS MQTT port, 1883. The client publishes the raw ADC value as a text payload to the `lab/ldr` topic.

The program first waits until the ESP32 is associated with the configured Wi-Fi network. It then configures the broker address and port. During each iteration, `analogRead(35)` acquires the raw ADC code, the value is printed to the serial monitor, and the MQTT connection is established if necessary. After `mqtt.loop()` services the client, the value is published and the program waits 1000 ms before the next

iteration. The 1000 ms delay is a minimum application delay rather than an exact sampling period because ADC acquisition, Wi-Fi processing, and MQTT operations add execution time.

B. Hardware and communication sequence

The complete data path is shown in Fig. 1. The sensor circuit produces an analog voltage, the ESP32 samples it, and the MQTT broker forwards the resulting payload to a subscribed client. Because `analogRead` returns an ADC code rather than a calibrated illuminance value, the reported payload is intentionally described as a raw LDR reading.

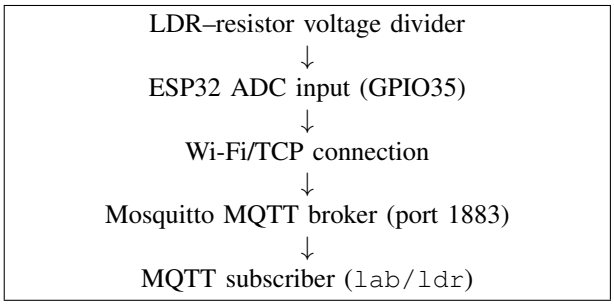

Fig. 1. CP2 block diagram for LDR acquisition and MQTT telemetry.

Fig. 2 summarizes the execution sequence. A failed MQTT connection is reported through the serial monitor and causes the current iteration to return before the next attempt.

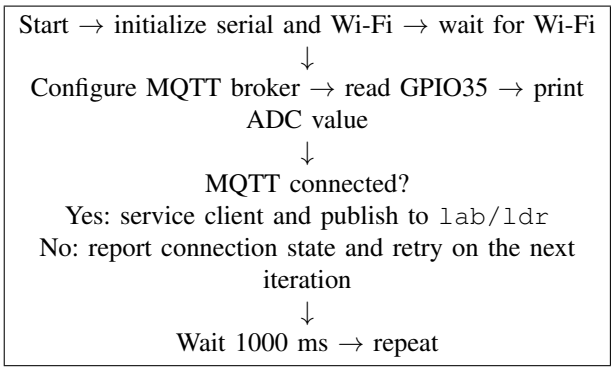

Fig. 2. CP2 program flow for acquisition and MQTT publication.

III. Development

The purpose of this section is to answer both the checkpoint and report questions of the lab, as well as discuss any relevant results obtained during the experiences.

The shared folder for checkpoint videos and code is available at LAB3 checkpoint materials. Individual demonstration videos are linked in the corresponding checkpoint sections.

A. CHECKPOINT 1: Previous Installations

1) **Arduino Installation and ESP32 Setup**
First, we downloaded and installed the Arduino IDE for our computer's operating system, which in this case was Windows 11, as shown in Figure 3.

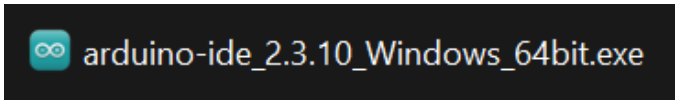

Fig. 3. Arduino IDE installation for Windows 11.

Once installed and opened, we needed to add support for the ESP32 board. To do this, we went to File > Preferences. There, we added the URL https://dl.espressif.com/dl/package_esp32_index.json in the "Additional Boards Manager URLs" field and clicked "OK", as shown in Figure 4.

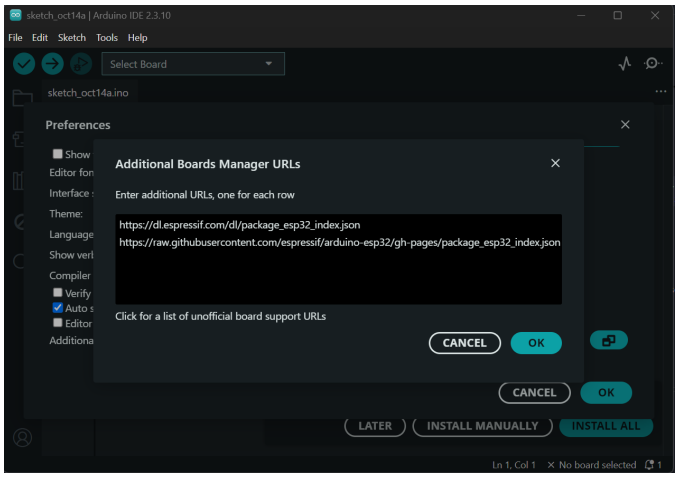

Fig. 4. Arduino IDE preferences with the ESP32 board manager URL added.

Then, we needed to install the board support package for the ESP32. To do this, we went to Tools > Board > Boards Manager. There, we searched for "esp32" and installed the package provided by Espressif Systems, as shown in Figure 5.

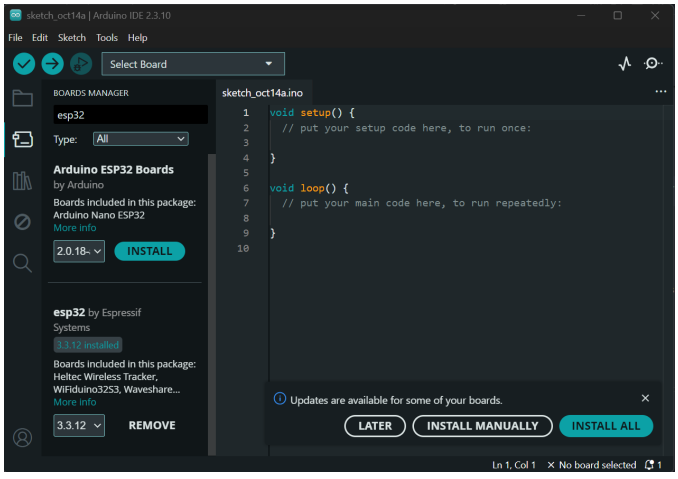

Fig. 5. Installation of the ESP32 board support package by Espressif Systems.

To finish the initial setup, we went to Select Board > Select Other Board and Port. There, we searched for "NodeMCU-32S" in the board selection section and

selected “COM5 Serial Port” in the ports section, as shown in Figure 6.

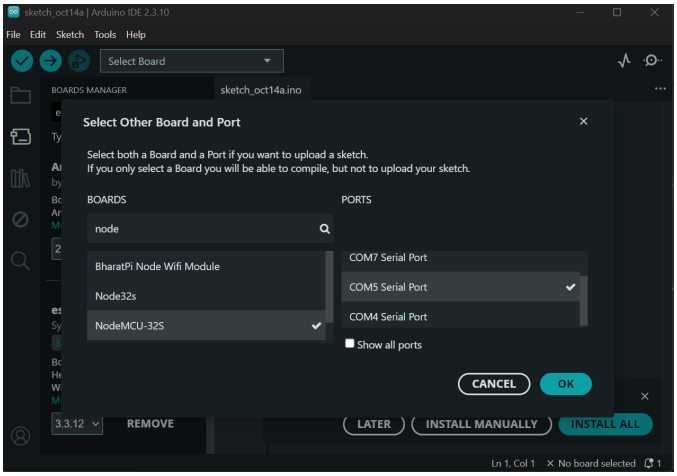

Fig. 6. Initial selection of the NodeMCU-32S board and COM5 serial port in Arduino IDE.

2) **Test Code Upload**

We tested a very simple code and actually ran into a small problem during the upload process. As shown in Figure 7, there was an issue with the selected serial port. We solved this by changing “COM5 Serial Port” to “COM3 Serial Port (USB)”, which corresponded to the connected ESP32, as shown in Figure 8. After fixing this issue, we successfully uploaded and ran the test code on the ESP32, as shown in Figures 9 and 10.

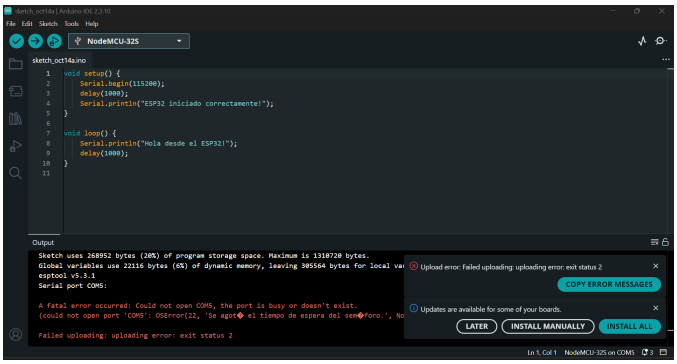

Fig. 7. Error encountered during the initial code upload due to the incorrect serial port selection.

3) **Mosquitto Installation**

First, we downloaded the Mosquitto executable installer for Windows, as shown in Figure 11. As instructed, Mosquitto was installed outside the Program Files directory. We also needed to add its installation path to the system environment variables in order to execute Mosquitto commands directly from the terminal. We then confirmed that Mosquitto could be executed successfully, as shown in Figure 12.

Then, we modified the Mosquitto configuration file, as shown in Figure 13, and successfully ran the broker using the updated configuration, as shown in Figure 14.

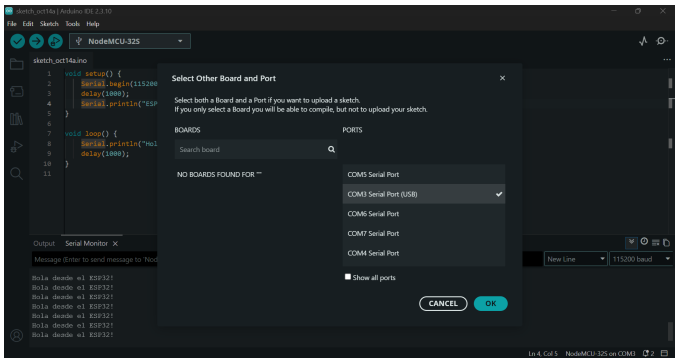

Fig. 8. Correction of the serial port selection from COM5 to COM3 (USB).

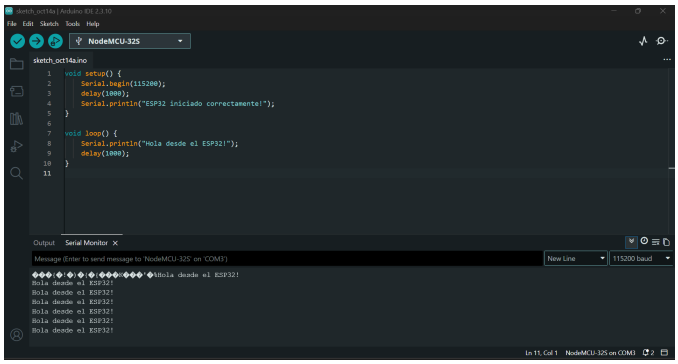

Fig. 9. Successful compilation and upload of the test code to the ESP32.

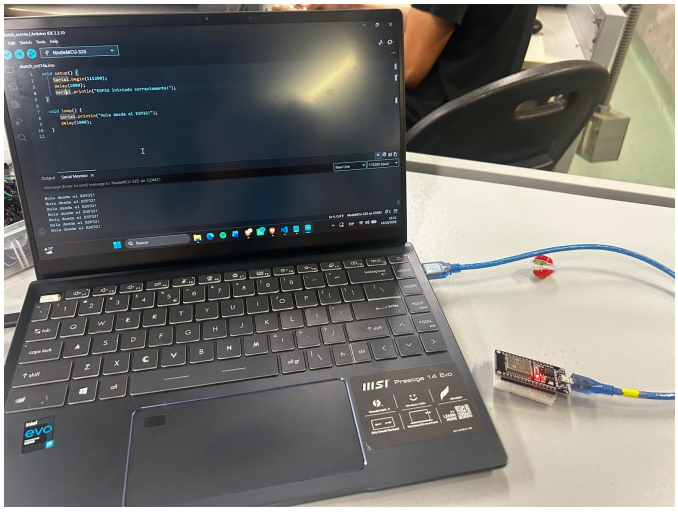

Fig. 10. Arduino IDE Serial Monitor displaying the messages generated by the ESP32 test code.

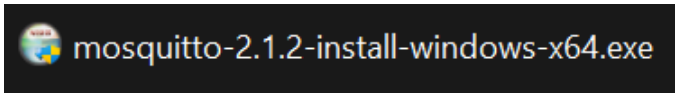

Fig. 11. Download of the Mosquitto installer for Windows from its official website.

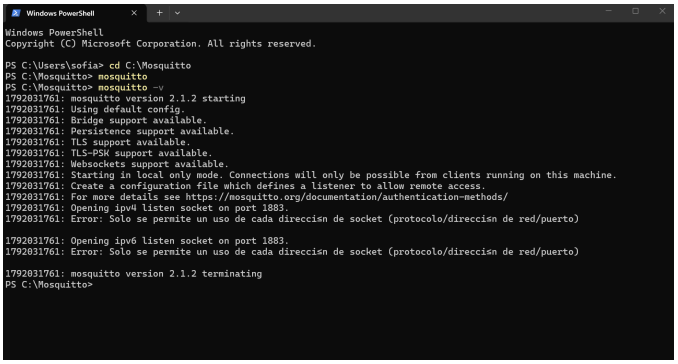

Fig. 12. Verification of the Mosquitto installation through the Windows terminal.

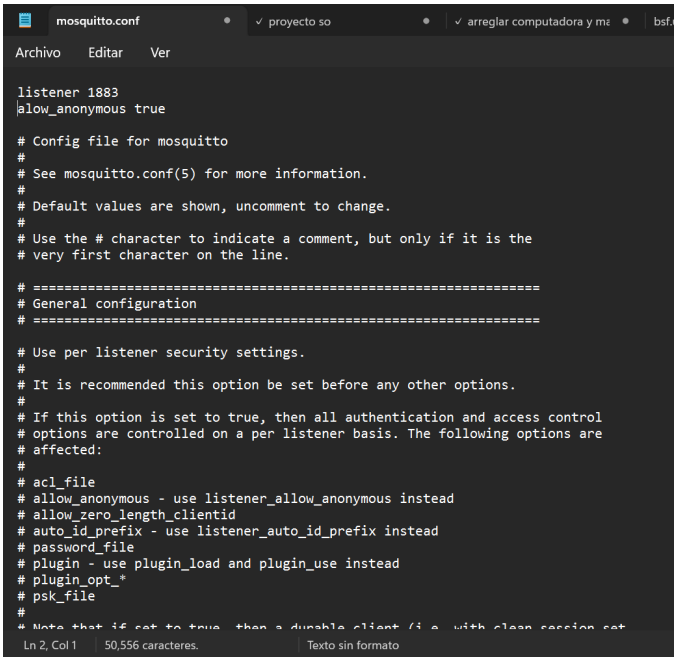

Fig. 13. Modification of the Mosquitto configuration file to enable connections on port 1883 and allow anonymous clients.

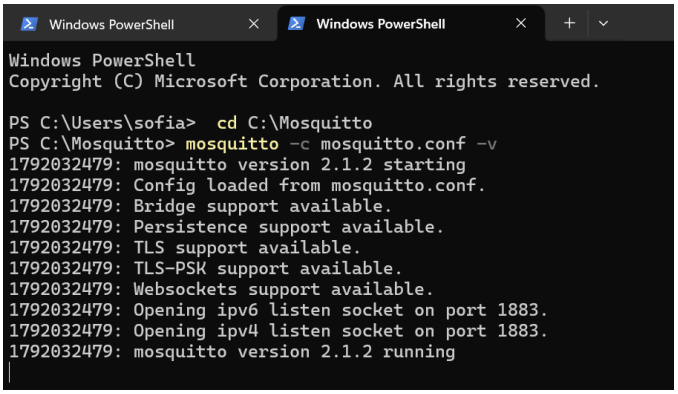

Fig. 14. Successful execution of the Mosquitto MQTT broker using the updated configuration file.

Finally, we tested MQTT communication by subscribing to the topic "test/topic" and publishing a test message to the same topic. As shown in Figure 16, we published a message, and as shown in Figure 15, we were able to receive it successfully through the subscribed client.

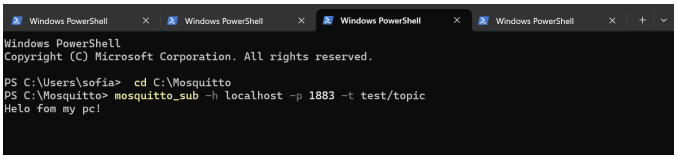

Fig. 15. MQTT subscriber receiving the test message through the Mosquitto broker.

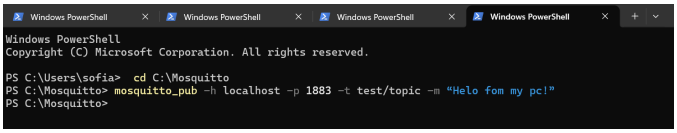

Fig. 16. Publication of a test message to the MQTT topic using the Mosquitto command-line client.

B. CHECKPOINT 2: Basic Implementation with Reading

The second checkpoint implemented a complete telemetry path from an LDR connected to the ESP32 ADC to an MQTT subscriber on another computer. The circuit was assembled as a voltage divider, and its output was connected to GPIO35.

1) Implementation Procedure: The implementation followed these steps:

1) The LDR voltage-divider output was connected to GPIO35, while the circuit was supplied from the ESP32-compatible voltage rail and ground. The fixed-resistor value and the physical orientation were recorded from the assembled circuit because they determine the ADC response.
2) The ESP32 board support package and the PubSubClient library were enabled in Arduino IDE. The board and the serial port corresponding to the connected ESP32 were selected.
3) The broker was configured on port 1883 and the subscriber was configured to listen to `lab/ldr`.
4) The sketch was compiled and uploaded to the ESP32. The serial monitor was opened at 115200 baud to verify Wi-Fi association, display the assigned IP address, and observe the ADC readings.
5) A Mosquitto subscriber was started before or during the test. The ESP32 then read the ADC, connected to the broker when necessary, and published each reading as a text payload.
6) The illumination over the LDR was changed while the serial output and the MQTT subscriber were observed. This test verifies that the acquisition path and the publication path respond to the same physical stimulus.

Figure 17 shows an excerpt of the ESP32 program used for the implementation. The code integrates the analog reading of the LDR with the Wi-Fi and MQTT communication functions.

In this way, the ESP32 is responsible for both acquiring the sensor data and publishing it to the broker, without requiring an additional microcontroller.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include "secrets_local.h"

WiFiClient cliente;
PubSubClient mqtt(cliente);

// Wi-Fi and broker configuration in setup():
WiFi.begin(ssid, password);
mqtt.setServer(mqtt_broker, 1883);

// Acquisition and publication in loop():
int valor = analogRead(35);
Serial.println(valor);
mqtt.loop();
mqtt.publish("lab/ldr", String(valor).c_str());
delay(1000);
```

Fig. 17. CP2 code excerpt for LDR acquisition and MQTT publication. Network configuration is kept in a separate local header; the complete sketch also includes connection handling.

The physical implementation is shown in Figures 18 and 19. The LDR and fixed resistor form a voltage divider, allowing variations in light intensity to be represented as changes in voltage at the ESP32 analog input. This is necessary because the ESP32 measures voltage rather than light intensity directly. The breadboard was used to assemble the circuit and connect the sensor to the development board.

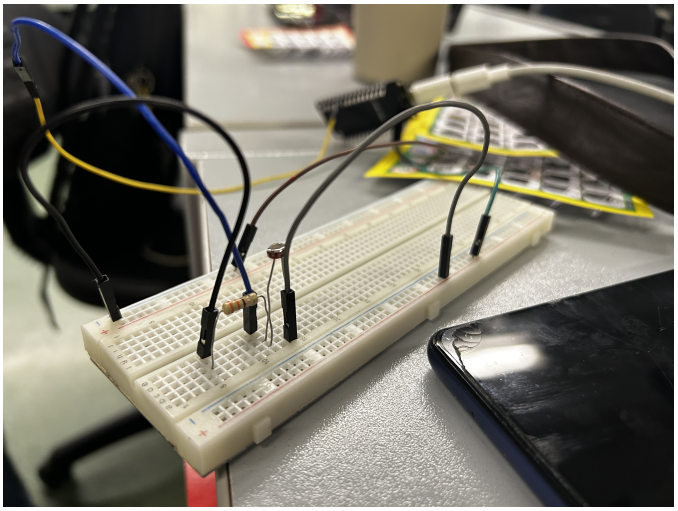

Fig. 18. LDR voltage-divider circuit assembled on a breadboard for analog light sensing.

2) Checkpoint Validation and Technical Discussion: The implementation was functionally validated through three main observations: the ESP32 acquired changing ADC readings, the broker received the published messages, and the MQTT subscriber displayed the transmitted values on the receiving computer.

As shown in Figure 20, the publisher-side terminal displayed a continuous sequence of LDR readings generated by the ESP32. This shows that the sensor was being read periodically, while the MQTT subscriber shown in Figure 21 confirms that these readings were also received on the other computer. The publication interval was approximately one second, although the actual time between messages also depends on the ADC, Wi-Fi, and MQTT operations.

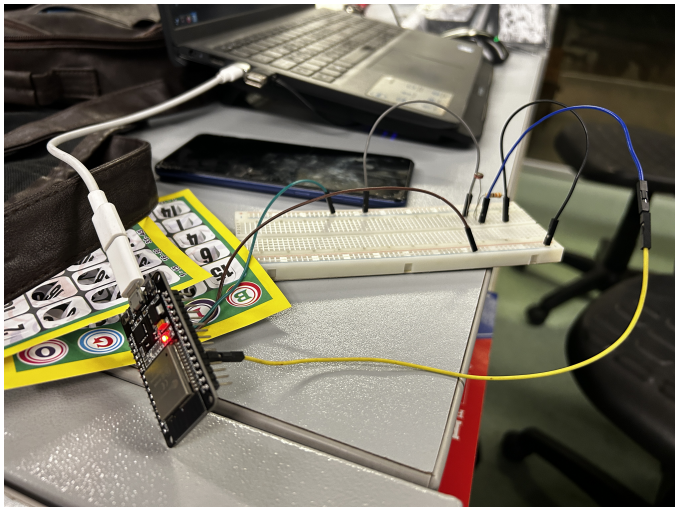

Fig. 19. Physical implementation of the LDR sensing circuit connected to the ESP32 development board.

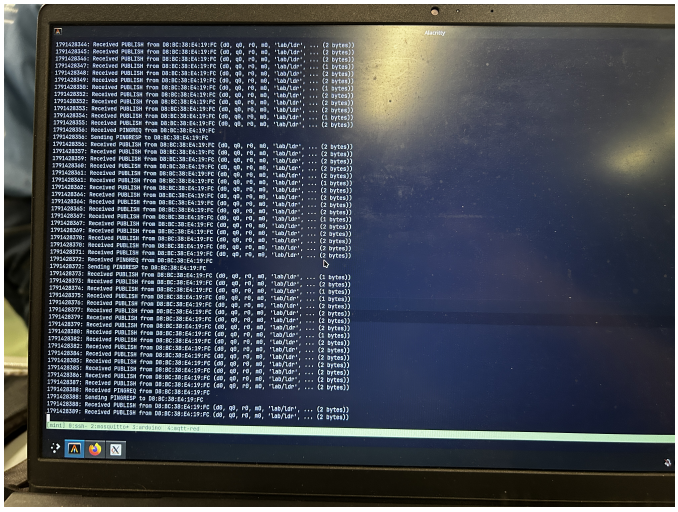

Fig. 20. Publisher-side terminal displaying the periodic LDR readings generated by the ESP32.

Figure 21 shows the MQTT subscriber receiving messages under the topic `lab/ldr`. The displayed values changed as the LDR was covered and uncovered, showing that variations in the physical input were reflected in the received data. This is important because it verifies not only that the devices were connected, but also that the information acquired by the sensor reached the other computer through the broker.

The two-computer communication was tested while the illumination reaching the LDR was manually modified. By covering and uncovering the sensor, the received values change accordingly, providing visual evidence that the system responds to changes in the environment and communicates the updated information remotely. The variation in the readings can be followed in greater detail in the CP2 demonstration video. This confirms the basic functionality of the MQTT publisher-subscriber architecture. The readings from the LDR

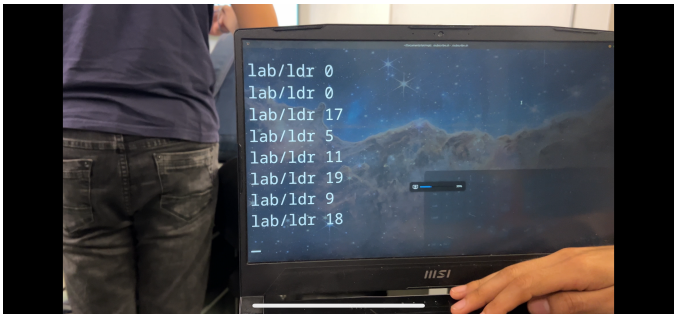

Fig. 21. MQTT subscriber displaying changing LDR readings received under the lab/ldr topic.

respond to changes in illumination. The implementation uses the default publication behavior of PubSubClient, MQTT 3.1.1 with QoS 0. Consequently, receiving the messages demonstrates successful communication during the test, but does not guarantee that every published message was delivered or provide a measurement of packet loss.

Checkpoint 2 demonstration video.

3) Block Diagram and Flowchart: The system architecture and the program execution sequence are represented in Figures 22 and 23, respectively.

The block diagram identifies the main components involved in data acquisition and transmission. The LDR provides the analog input to the ESP32, which processes the reading and publishes it through Wi-Fi to the Mosquitto broker. The broker then forwards the message to the subscribed client on the second computer.

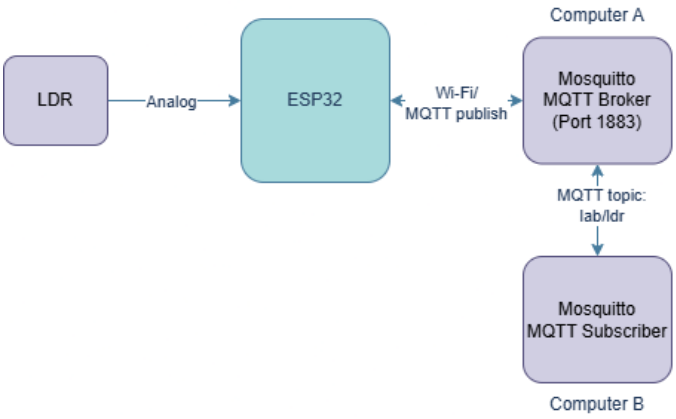

Fig. 22. Block diagram of the LDR sensing and MQTT communication system using an ESP32.

The flowchart describes the internal execution of the ESP32 program, including initialization, sensor acquisition, MQTT connection verification, and periodic publication. The connection decision is relevant because the ESP32 must establish communication with the broker before it can successfully publish the acquired data.

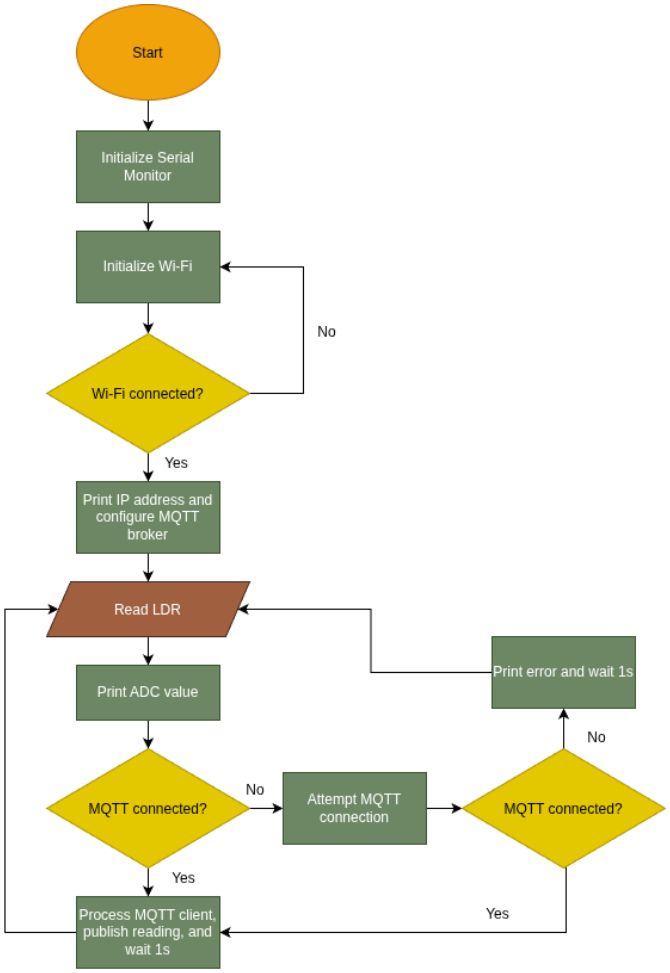

Fig. 23. Flowchart of the ESP32 program for periodic LDR readings and MQTT publication.

C. CHECKPOINT 3: Smart Ventilation and Light Control with MQTT

In this checkpoint, we implemented an automated system that responds to changes in ambient light using an ESP32, an LDR, and a potentiometer. The LDR measures the light level, while the potentiometer establishes an adjustable threshold. Depending on the comparison between these values, the system controls a servomotor, a buzzer, and LED indicators. Additionally, MQTT communication allows the servomotor to be controlled remotely through manual commands.

1) Physical Implementation: The system was assembled using an ESP32, an LDR with a fixed resistor, a potentiometer, an SG90 servomotor, a buzzer, three LEDs (red, green, and blue), resistors, a breadboard, and jumper wires. The readings from the LDR and the potentiometer are compared to determine whether the measured light level exceeds the threshold selected by the user. When the threshold is exceeded, the servomotor moves $90°$ to simulate closing a blind, the red LED turns on, and the buzzer should activate as an audible warning. Otherwise, the servomotor returns to its open position $0°$, the green LED turns on, and the buzzer remains off. The blue LED indicates the connection to the MQTT broker.

As shown in Figure 24, the red LED is illuminated while

the blue LED remains on. This is consistent with the system entering its alert state when the measured light exceeds the selected threshold. The red LED indicates that the automatic control should close the blind, while the blue LED represents the MQTT connection status.

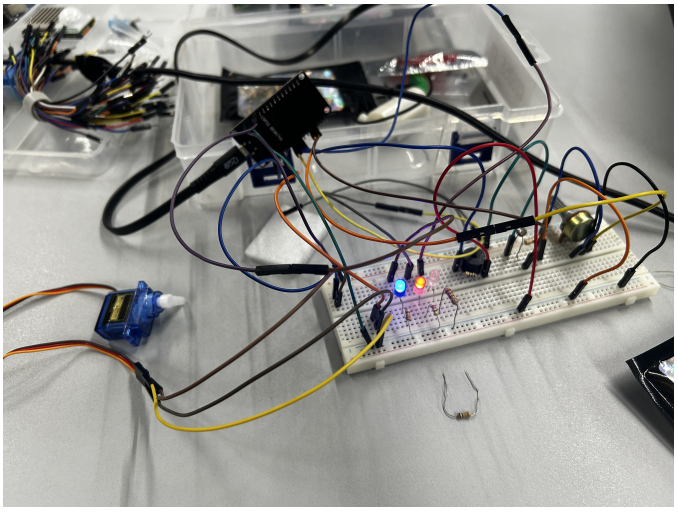

Fig. 24. Physical implementation of the automatic light control system with the red and blue LEDs activated.

In Figure 25, the green LED is illuminated instead of the red LED. This corresponds to the normal operating state, in which the measured light does not exceed the configured threshold and the blind should remain open. Comparing both photographs shows how the LED indicators distinguish between the two operating conditions, while the blue LED remains illuminated in both cases.

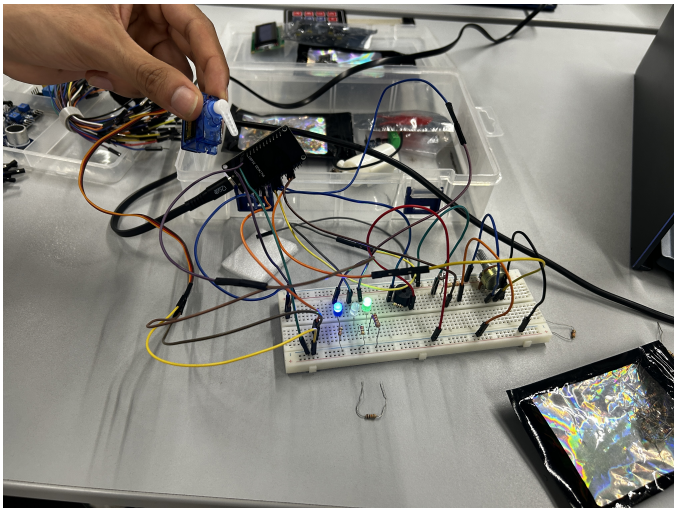

Fig. 25. Physical implementation showing the green and blue LEDs activated during normal operation.

2) Automatic Control: To test the automatic mode, we modified the threshold using the potentiometer, covered the LDR to change the light input and observed how the system responded to the LDR reading.

As shown in the demonstrations, the system responds correctly when the potentiometer is adjusted. When the light reading exceeds the selected threshold, the red LED turns on, the servomotor moves to the closed position, and the buzzer is expected to activate. When the light reading is below the threshold, the green LED turns on and the servomotor returns to the open position. This shows that the automatic control works as expected and responds to changes in the selected threshold. The sequence of sensor adjustments and actuator responses can be seen in greater detail in the automatic-control videos.

However, we noticed that the buzzer did not always sound immediately when activated. Sometimes it started working after a while, possibly due to a loose connection, although we could not confirm this since we did not have the opportunity to test another buzzer or replace the wires.

Automatic control demonstration 1

Automatic control demonstration 2

3) Manual Control through MQTT: Besides automatic operation, the system also supports remote control through MQTT commands. For this test, commands were manually published from a terminal connected to the MQTT broker, allowing us to change the servomotor position without depending on the current LDR reading.

During this demonstration, the command `cerrar` was sent to move the servomotor to the closing position, while `abrir` moved it back to the open position. The command `auto` was then used to return the system to automatic operation, allowing the servomotor to respond again to the comparison between the LDR reading and the potentiometer threshold. These transitions can be followed in greater detail in the manual-control video.

This test is relevant because it demonstrates that the ESP32 can operate as both an MQTT publisher and subscriber. While the sensor measurements and system status can be published to the broker, the ESP32 can also receive commands to modify the behavior of an actuator.

Manual MQTT control demonstration

Figures 26 and 27 show the MQTT communication during the demonstration. In Figure 26, the commands `abrir` and `cerrar` were published to the `iluminacion/control` topic to control the servomotor remotely. Figure 27 also shows the Mosquitto broker receiving PUBLISH messages on the `iluminacion/datos` topic, indicating that the ESP32 was transmitting telemetry while the system was operating. Together with the manual control video, these results show that MQTT communication was used for both monitoring and remote control. The live telemetry and its changes during operation can be seen in greater detail in the CP3 demonstration videos.

4) Block Diagram and Flowchart: The system architecture is represented in Figure 28. The block diagram identifies the sensing components, the ESP32 controller, the actuators, and the MQTT communication path.

The LDR and potentiometer provide analog inputs to the ESP32, which determines the operating condition and controls the servomotor, buzzer, and LEDs. Through Wi-Fi, the ESP32 communicates with the Mosquitto broker to publish telemetry and receive remote commands.

Fig. 26. Mosquitto terminal showing MQTT control messages used to change the servomotor position and operating mode.

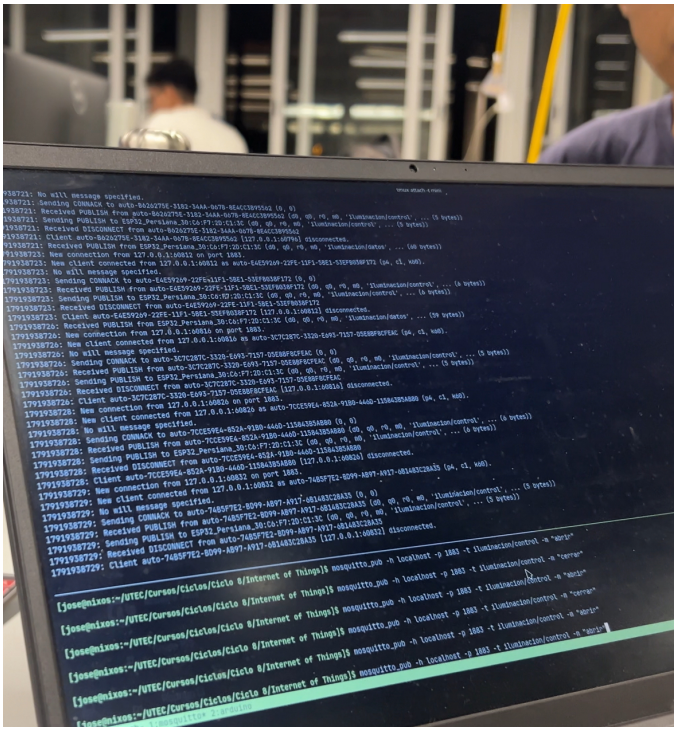

Fig. 27. Mosquitto broker logs showing MQTT client connections and message activity during the system demonstration.

The program execution sequence is shown in Figure 29. The flowchart includes the initialization of the ESP32, acquisition of the analog readings, verification of the MQTT connection, and selection between automatic and manual operation. In automatic mode, the system compares the LDR reading with the threshold and changes the actuator states accordingly. In manual mode, the servomotor responds to the received MQTT commands. This separation allows remote control while maintaining the option to return to autonomous operation.

D. CHECKPOINT 4: Arduino Mega + ESP32 Communication

In this checkpoint, we implemented a small system for simulating microcontroller communication and data transmission via UART and MQTT.

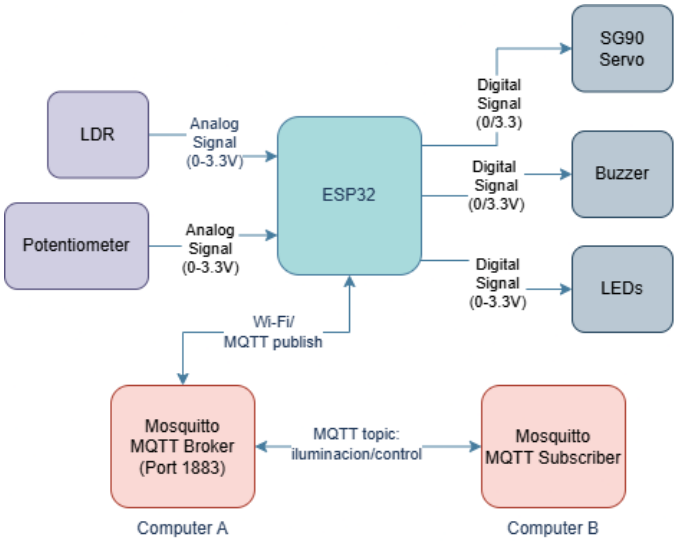

Fig. 28. Block diagram of the automatic light control system with local actuators and MQTT communication.

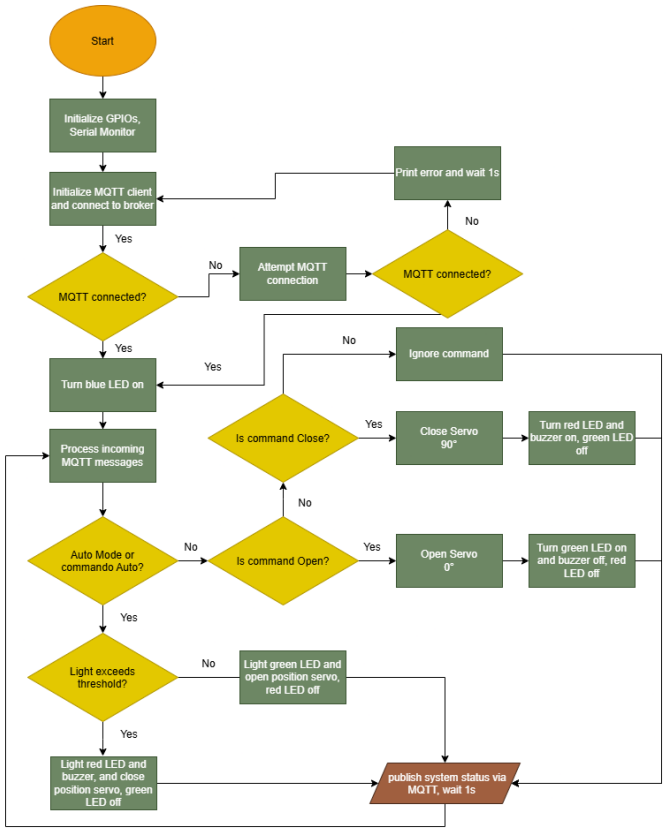

Fig. 29. Flowchart of the ESP32 program including automatic threshold-based control and manual MQTT commands.

After connecting the Arduino MEGA and ESP32 communication pins, we were able to transmit a fixed test message from the Arduino, receive it on the ESP32, and publish it to an MQTT topic, which we subscribed to from a laptop running both the broker and a Mosquitto client. A voltage divider adapts the Mega's $5\,\mathrm{V}$ TX signal to the ESP32's $3.3\,\mathrm{V}$ logic level.

1) Physical Implementation: To enable communication between these two microcontrollers, their serial communication pins must be connected to each other. As their GND reference must also be the same, we connected their ground pins as well.

- Arduino RX1: connected to ESP32 TX2
- Arduino TX1: connected to ESP32 RX2 (with v.divider)
- Arduino GND: connected to ESP32 GND

For the voltage divider, we used three $10\,\mathrm{k\Omega}$ resistors in series. With a $5\,\mathrm{V}$ input, the ideal node voltages relative to GND are $3.33\,\mathrm{V}$ between the first and second resistors and $1.67\,\mathrm{V}$ between the second and third resistors. In the schematic, the ESP32 RX input is connected to the first of these nodes, so the divider ratio is $20/(10+20) = 2/3$. These are calculated values, not measured voltages.

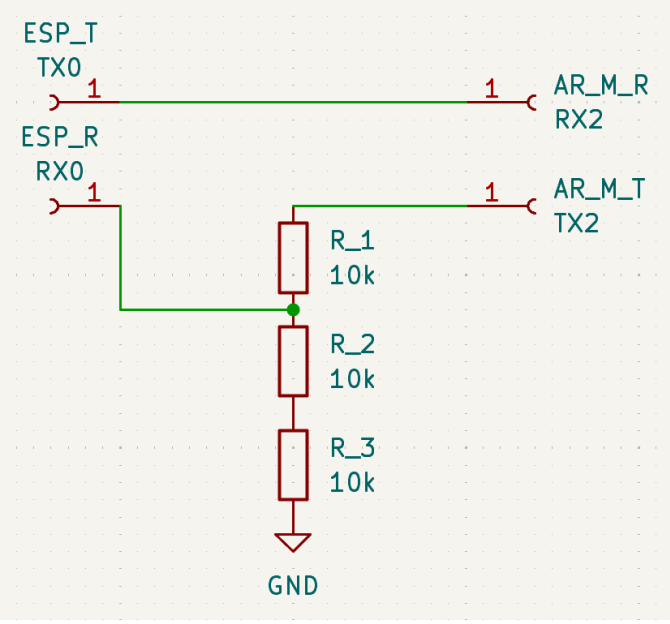

Fig. 30. UART connection and voltage-divider schematic.

The R and T used to name the pins stand for **receive** and **transmit**, which, despite being apparent, took us a minute to notice (sorry).

2) Arduino Code Upload: The Arduino Mega sketch included in `code/arduino.ino` sends a fixed test string via UART using `Serial1.println()` at 9600 baud. It uses `Serial1` rather than the USB serial interface, `Serial`. Each loop includes a 1000 ms delay; this is an application delay, not a measured transmission period.

3) ESP32 Code Upload: The ESP32 connects to a WiFi network, receives messages via UART, and publishes them as an MQTT client to the Mosquitto broker running on the laptop.

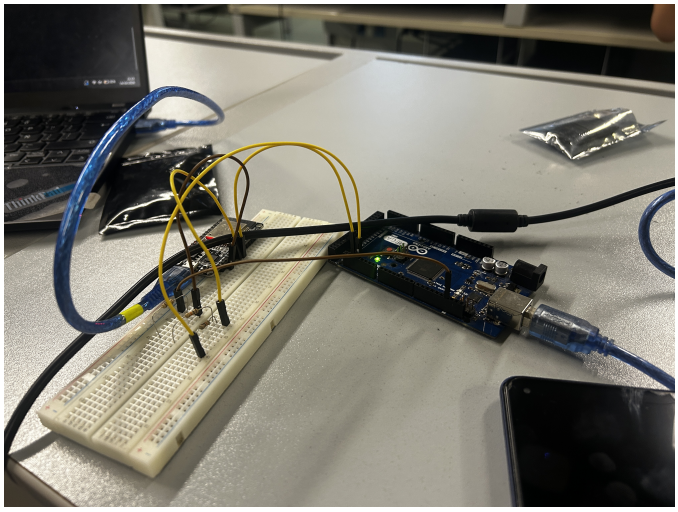

Fig. 31. Physical system with a voltage divider to avoid frying the ESP.

To make sure we didn't run into any issues with the WiFi network, we set up a mobile hotspot using cellular data. The SSID used was **quaso**.

In the setup function, besides initializing both the *Serial* and *Serial2* objects, we connect to the mobile hotspot, configure the destination broker address and port 1883 using `mqtt.setServer(...)` and then attempt to connect to that broker. This call configures the MQTT client; it does not start an MQTT server on the ESP32.

```
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  WiFi.begin(ssid, password);
  Serial.println(WiFi.localIP());
  mqtt.setServer(mqtt_server, 1883);

  while (!mqtt.connected()) {
    Serial.print("Connecting to MQTT...");

    if (mqtt.connect("esp32cito")) {
      Serial.println("connected!");
    } else {
      Serial.print("failed, state=");
      Serial.println(mqtt.state());
      delay(2000);
    }
  }
}
```

Fig. 32. ESP32 code setup

After the setup, the ESP32 reads a newline-terminated string from `Serial2` and publishes non-empty messages to `hey/guys`. If the broker connection fails, the sketch retries in a blocking loop before servicing UART reception.

4) MQTT Monitoring: After flashing both the Arduino and ESP, and successfully setting up the config file in checkpoint 1, we ran the Mosquitto broker and, on a separate terminal session, subscribed to the MQTT topic with the following commands:

- `./mosquitto -c ./mosquitto.conf -v`
- `./mosquitto_sub -h localhost -p 1883 -t hey/guys`

Here, `-h` selects the broker host, `-p` its port, and `-t` the subscription topic. The use of `localhost` is appropriate because this subscriber runs on the same laptop as the broker; a subscriber on another computer must use the broker's reachable address instead.

Then, we powered on both microcontrollers, and watched how the laptop received the messages sent by the ESP32.

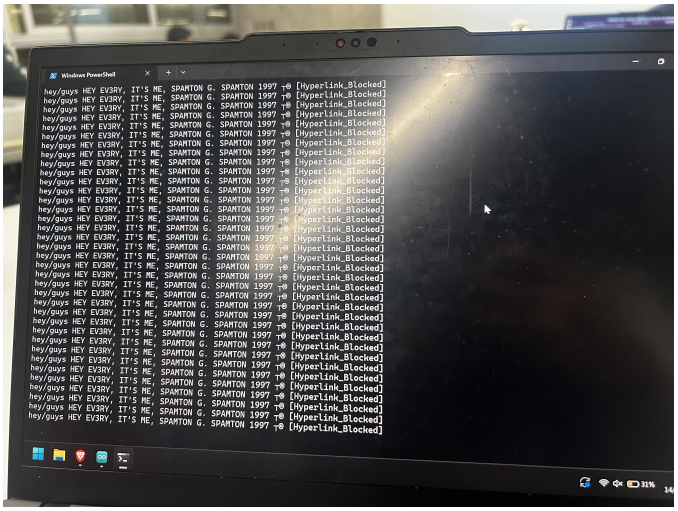

Fig. 33. Messages sent by ESP32 shown on the Mosquitto client

IV. Conclusions and Recommendations

This laboratory helped us understand how to connect a physical circuit to a network and monitor or control it from another computer. Installing Mosquitto was straightforward, but having the broker running was not enough: the ESP32 also needed the correct laptop address and port, and the firewall had to allow the connection. We therefore recommend checking these settings before looking for errors in the sensor or MQTT code. The LDR readings and the blind-control experience showed how the same communication model can support both monitoring and commands, with the automatic and manual behavior demonstrated in the videos. The intermittent buzzer response also reminded us to test each component separately when the rest of the system appears to work. One of the most useful lessons from connecting the Mega to the ESP32 was that sharing a communication protocol does not mean sharing the same voltage levels: the Mega's TX signal needs a voltage divider before reaching the ESP32 RX input to avoid damaging it. We recommend checking the divider connections and measuring its output before connecting the boards. Finally, because the ESP32 needs network credentials to reach the broker, the Wi-Fi password should be kept in a separate local configuration file excluded from the shared code. This makes it easier to share the programs and reproduce the checkpoints without accidentally exposing credentials.

References

[1] *MQTT Version 3.1.1*, OASIS, Oct. 2014, oASIS Standard, 29 de octubre de 2014. Consultado el 14 de octubre de 2026. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[2] *Arduino-ESP32: ADC API*, Espressif Systems, documentación oficial, funciones analogRead y analogReadMilliVolts. Consultada el 14 de octubre de 2026. [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html

[3] N. O'Leary, "PubSubClient, versión 2.8: código fuente," 2020, pubSubClient.h y PubSubClient.cpp, MQTT_VERSION, publish y loop. Consultados el 14 de octubre de 2026. [Online]. Available: https://github.com/knolleary/pubsubclient/tree/v2.8/src

[4] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.

[5] D. Thangavel, X. Ma, A. Valera, H. X. Tan, and C. K.-Y. Tan, "Performance Evaluation of MQTT and CoAP via a Common Middleware," in *2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing*, 2014, pp. 1–6.

[6] D. Borsatti, W. Cerroni, F. Tonini, and C. Raffaelli, "From IoT to Cloud: Applications and Performance of the MQTT Protocol," in *2020 22nd International Conference on Transparent Optical Networks (ICTON)*, 2020, pp. 1–4.

[7] U. Hunkeler, H. L. Truong, and A. Stanford-Clark, "MQTT-S — A publish/subscribe protocol for Wireless Sensor Networks," in *2008 3rd International Conference on Communication Systems Software and Middleware and Workshops (COMSWARE)*, 2008, resumen disponible en IBM Research. Consultado el 14 de octubre de 2026. [Online]. Available: https://research.ibm.com/publications/mqtt-s-a-publishsubscribe-protocol-for-wireless-sensor-networks

[8] F. Palmese, A. E. C. Redondi, and M. Cesana, "Adaptive Quality of Service Control for MQTT-SN," *Sensors*, vol. 22, no. 22, p. 8852, 2022.

[9] S. Jeddou, F. Fernández, L. Diez, A. Baina, N. Abdallah, and R. Agüero, "Delay and Energy Consumption of MQTT over QUIC: An Empirical Characterization Using Commercial-Off-the-Shelf Devices," *Sensors*, vol. 22, no. 10, p. 3694, 2022.